

Homework 4: Privacy and Unlearnable Examples

李玮博 2400013080

2026-05-29

1. 作业要求

本次作业包含两部分：Membership Inference Attack (MIA) 和 Unlearnable Examples。PPT 要求 MIA 在 MNIST 上达到超过 85% 的攻击准确率；Unlearnable Examples 需要分别完成 class-wise 与 sample-wise 的 min-min 生成过程，并使模型在生成数据集上训练后对于干净测试集的最终分类准确率低于 40%。提交内容为报告和代码，不提交 checkpoint。

2. MIA Attack

2.1 数据划分

MIA.ipynb 中的 `custom_MNIST` 将 MNIST 拆成 target model 和 shadow model 两条数据流。target model 使用前 25000 张训练图作为成员样本，前 5000 张测试图作为非成员样本。shadow model 使用训练集后 25000 张和测试集后 5000 张构造多个影子训练/验证集合，用它们生成攻击模型的训练数据。

影子测试集的切片修正为每个 shadow model 取 500 张验证样本，与 `__getitem__` 中非 target 验证集的 `index % 500` 保持一致，避免标签和图像切片长度不一致。

2.2 Target 与 Shadow 模型训练

target model 和 shadow model 都使用 `Net_mnist`，优化器采用 SGD，学习率为 0.001，momentum 为 0.5，训练 40 个 epoch。训练函数会保存验证准确率最好的模型权重。训练结束后，再用该最佳模型分别遍历训练集和验证集，生成 MIA 特征与成员标签：

$$y_{mia} = 1 \quad \text{if sample is in training set,} \quad y_{mia} = 0 \quad \text{if sample is in validation set.}$$

这样可以避免在最后一个 epoch 中一边更新参数一边提取特征，从而使攻击训练集和 target 评估集都来自稳定的模型状态。

2.3 攻击特征

攻击模型不接触原始图像，只使用被攻击分类器的输出行为。对每个样本，代码从 logits 中提取以下特征：

- 原始 softmax 概率向量；
- 降序排列后的 softmax 概率向量；
- 真实类别对应置信度；
- 预测分布熵；
- 逐样本交叉熵损失；
- 第一大和第二大置信度的 margin；
- 分类是否正确。

成员样本通常具有更高真实类别置信度、更低 loss、更低 entropy，这些统计量可以帮助攻击分类器区分训练集样本和非训练集样本。

2.4 攻击分类器

Notebook 优先使用 LightGBM:

```
lgb.LGBMClassifier(  
    objective="binary",  
    reg_lambda=10,  
    n_estimators=1000,  
    learning_rate=0.03,  
    num_leaves=31,  
)
```

如果运行环境没有安装 lightgbm, 代码会自动退回到 `sklearn.ensemble.HistGradientBoostingClassifier`, 保证 notebook 仍可执行。最终使用 `accuracy_score(label_test_set, y_pred_lgbm)` 计算 MIA 准确率; 原始 notebook 中用成员样本数作为分母的写法已修正为使用总样本数。

3. Unlearnable Examples

3.1 Min-min 思路

Unlearnable Examples 的目标是在训练图片上加入有界扰动 δ , 使模型在训练时容易学习到扰动对应的捷径, 而不是图像中真正可泛化的语义特征。扰动约束为:

$$\|\delta\|_{\infty} \leq \epsilon, \quad \epsilon = 16/255.$$

代码使用 min-min 更新: 先固定扰动训练当前 base model 若干步; 再固定 base model, 对输入图像执行使交叉熵下降的 PGD 更新。由于更新方向是最小化训练损失, 扰动会变成模型容易拟合的 shortcut。

3.2 Class-wise 噪声

Classwise.ipynb 维护形状为 [10, 3, 32, 32] 的 noise, 即每个 CIFAR-10 类别共享一张扰动。每轮外循环包含三步:

1. 用当前类别噪声加到 batch 图像上, 训练 base_model 10 个 batch。
2. 冻结模型参数, 对整个训练集运行 `noise_generator.min_min_attack`。
3. 将同一类别样本得到的 image-wise eta 求平均, 更新该类别共享扰动。

生成最终数据集时, 代码按照每张图的真实标签取对应类别噪声, 转换到像素尺度后加到原图, 再 clip 到 [0,255]。可视化部分也改为按标签索引类别噪声, 而不是直接用样本下标索引 noise。

3.3 Sample-wise 噪声

Samplewise.ipynb 维护形状为 [50000, 3, 32, 32] 的 noise, 即每个训练样本拥有自己的扰动。因为训练集 dataloader 不 shuffle, 代码可以用 batch 的起止下标精确定位每张图对应的扰动。每轮外循环同样先训练 base model, 再对每个 batch 生成新的 eta 并写回 `noise[batch_start:batch_end]`。

构造 unlearnable dataset 时, 扰动先乘以 255 再添加到原始 uint8 图像。这里保留扰动的正负号, 只对加噪后的图像做 clip; 如果在加图像前把扰动本身 clamp 到 [0,255], 负扰动会丢失, 影响 min-min 结果。

3.4 评估

两个 notebook 都用生成后的 unlearnable training set 重新训练一个 ResNet-18, 并在干净 CIFAR-10 test set 上评估。作业目标是最终 Clean Accuracy 低于 40%。代码中保留了 `max_outer_loops`, 用于防止在 CPU 或资源不足环境中无限循环; 在 GPU 环境中如果还没有达到停止条件, 可以继续增加外循环。

4. 代码与提交内容

本次提交包含:

- MIA.ipynb / MIA.py: MIA 数据划分、target/shadow 训练、攻击特征提取和攻击分类器;

- `Classwise.ipynb` / `Classwise.py`: class-wise min-min unlearnable examples;
- `Samplewise.ipynb` / `Samplewise.py`: sample-wise min-min unlearnable examples;
- `model.py`、`util.py`、`models/`: MNIST 与 CIFAR-10 模型和工具代码;
- `requirements.txt`: 运行依赖说明。

代码修复了硬编码 GPU 调用和硬编码数据路径: CIFAR-10 数据目录默认使用 `./data`, 也可以通过环境变量 `CIFAR10_ROOT` 指定; 模型和张量统一使用 `device`, 可在 CUDA 或 CPU 环境运行。`models/__init__.py` 中的 `mlconfig` 注册也改为可选依赖, 避免环境未安装 `mlconfig` 时无法导入 `models.ResNet`。